

CS561 Final Report: Implementing SIEVE in the Postgres Bufferpool

Ryan Crosier
Boston University

Will Garlington
Boston University

Jackson Gilstrap
Boston University

Abstract

The buffer cache is a key data structure in the DBMS that helps reduce unnecessary disk I/Os. Eviction policies govern the removal of stale or infrequently accessed data, making room for new entries. Some of the most common eviction algorithms are Least Recently Used (LRU), Least Frequently Used (LFU), and First In First Out (FIFO). PostgreSQL uses CLOCK Sweep [1] (Figure 1), a modification of the LRU algorithm designed to minimize the overhead of maintaining an ordered list of recently used pages. For our project, we implement SIEVE, an eviction policy originally designed as an LRU alternative for web caches [5] into PostgreSQL. SIEVE is advantageous due to its lock-free design and its ability to be easily integrated into existing cache architectures. Whilst SIEVE was originally made for the web, there may be interesting use for it in database systems such as PostgreSQL. Our experiments benchmark SIEVE against other eviction policies using *pgbench*. We find that SIEVE provides little improvement to the cache-hit ratio over the default buffer cache eviction policy on mixed and skewed workloads; however, minor latency improvements are observed.

1 Introduction

1.1 Motivation

The most successful cache eviction policy introduced in recent years is SIEVE [5], an algorithm which is even simpler to implement than LRU. While originally designed for web-caching workloads, it has been found to be useful in many workloads with a high degree of data skew. While the paper introducing SIEVE concedes that it may not perform well for buffer cache workloads due to the presence of range scans, many real-world workloads may have a much higher proportion of point queries for which the benefits of SIEVE can be exploited.

1.2 Problem Statement

The problem we explored is whether or not there are workloads for which SIEVE outperforms other simple eviction policies for the bufferpool in PostgreSQL. By default PostgreSQL uses the CLOCK algorithm for buffer cache eviction, we also implemented LRU as another point of comparison. We then explored workloads with varying data skews to determine whether SIEVE would grant performance benefits or reduce write amplification.

1.3 Contributions

We make the following contributions:

- We implemented a framework in PostgreSQL for implementing and switching between different buffer cache eviction policies at compile time. The policies we implement in addition to CLOCK are SIEVE and LRU.

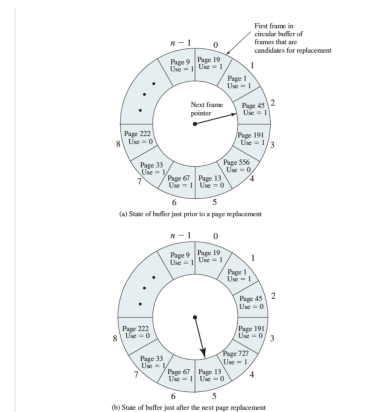


Figure 1: Clock Sweep diagram. [2]

- We present results indicating that SIEVE does not offer any significant performance benefits over CLOCK in skewed and mixed workloads for point queries.

2 Background

For decades, cache eviction has been primarily dominated by primitive algorithms such as First In First Out (FIFO) and Least Recently Used (LRU). More complex algorithms such as Adaptive Replacement Cache (ARC) [3] typically have better results by exploiting different parts of the system to account for weaknesses in the aforementioned algorithms; however require the developer to implement hundreds of lines of code. This persistent trade-off between ease of implementation and performance has been continuously discussed and researched in system research. As discussed in recent work, [5] SIEVE is a recent advancement in this space, originally designed for and evaluated against web cache workloads. The authors categorize SIEVE as a cache eviction primitive similar to LRU or FIFO. Structurally, it is built upon a single FIFO queue and utilizes a "moving hand" pointer to track whether data has been recently accessed. This hand scans from the tail toward the head of the queue, resetting the "visited" bit of objects it encounters. An object is only evicted if its visited bit is 0; otherwise, it is retained in its original position, effectively "sifting" the cache to preserve popular data.

2.1 Simplicity and Performance

A primary advantage of SIEVE is its ease of integration; the authors report that it can be implemented within existing production environments using an average of fewer than twenty lines of code changes. This simplicity translates directly to high-performance scalability. In multi-threaded environments, SIEVE has demonstrated nearly double the throughput of optimized LRU implementations.

3 Architecture

3.1 Buffer Policy Framework

To facilitate the testing of several eviction policies in a common 'environment', we've implemented an eviction policy framework on which all policies can be built. This framework is made up of two primary components: a `BufferStrategyCommon` which holds a buffer lock and other policy data, and an `EvictionVtable` which defines virtual functions to be implemented by new policies. Because *freelist.c*, where eviction logic is implemented, does not dictate when or how buffers enter or leave the buffer pool, our virtual functions can focus purely on the mechanics of evictions and insertions.

3.2 Sieve Implementation

Our SIEVE implementation was designed with static arrays to minimize the overhead of memory reallocation. Two static arrays sized to N_{Buffers} , the number of buffers allowed in the pool, combine to recreate the behavior of a linked list. Our implementation does not have the lockless hit described in the paper, but does utilize its idea of an unchanging doubly linked list with $O(1)$ access. Each insertion or *GetBuffer* (find victim) call is treated as an atomic action where a Spinlock is held until the function completes. After every insertion or deletion, the ID of the buffer is used to access the array and either mark the buffer as the new head (insertion) or designate the buffer as unreachable (eviction).

4 Benchmarking & Results

We have made substantial changes to our benchmarking setup since the midway report. These changes will be pointed out in the following subsections. We would also like to mention that the results from our midway report are entirely invalid because we had not been properly switching cache eviction policies due to a miscommunication. This has been resolved, and detailed information on how to test this yourself can be found in our codebase.

4.1 Hardware

We perform all of our evaluations on a system running Linux Mint (a Debian fork). The system has an AMD Ryzen 5 5600X CPU with 6 Cores at 3.7GHz base clock, and 32 GB of DDR4 memory. The system is equipped with two storage devices: a 256 GB SATA SSD and a 1 TB NVMe SSD.

4.2 Postgres Setup

Postgres data is stored on the NVMe SSD for every test, and the WAL is stored on the SATA SSD. In the midterm, the only device was the SATA SSD. Benchmarks use the standard `pgbench` tables with a scale factor of 1000. In our midterm results, we had erroneously used a scale factor of 1 when generating the tables. Auto-vacuuming is disabled, and the tables are vacuumed manually after each test to ensure a clean database for the next test. In our midterm results, auto-vacuuming had been enabled.

4.3 Pgbench Workloads

We test over various workloads with different data localities and read/write ratios. For a skewed workload, we draw points from a

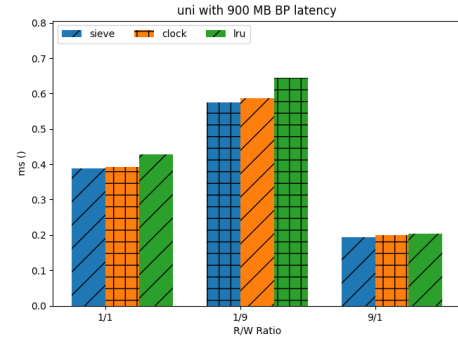


Figure 2: Average query latency of uniform queries with a 900MB BP.

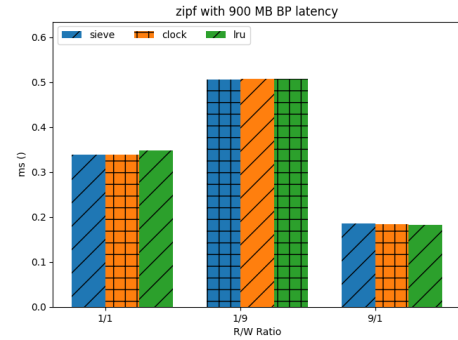


Figure 3: Average query latency of skewed Zipfian queries with a 900MB BP.

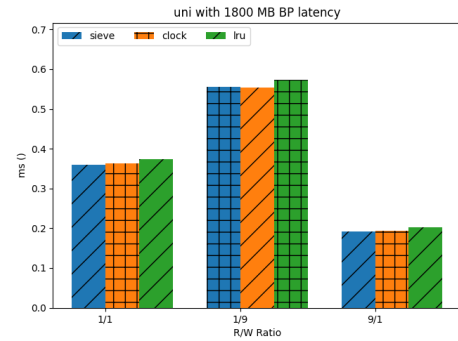


Figure 4: Average query latency of uniform queries with a 1800MB BP.

Zipfian distribution with $\alpha = 1.01$. This is a lower parameter than we used in the midterm to get a slightly more dramatic skew.

4.4 Results

Our results show that in the SQL use case we tested, SIEVE does show signs of slightly improved performance for very small buffers, but this difference in performance disappears as buffer size increases.

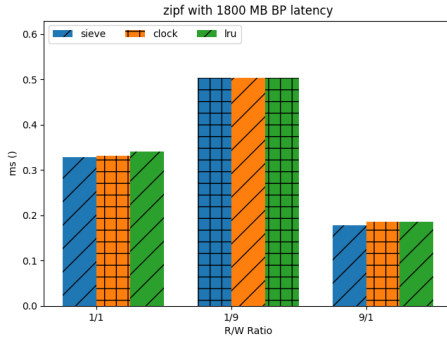


Figure 5: Average query latency of skewed Zipfian queries with a 1800MB BP.

Distribution	R/W Ratio	CLOCK Latency (ms)	SIEVE Latency (ms)	SIEVE Speedup %	LRU Latency (ms)	LRU Speedup %
Uniform	1/1	0.392	0.388	1.031	0.428	-8.411
Uniform	1/9	0.586	0.575	1.913	0.645	-9.147
Uniform	9/1	0.199	0.193	3.109	0.203	-1.970
Zipfian	1/1	0.338	0.339	-0.295	0.348	-2.874
Zipfian	1/9	0.508	0.506	0.395	0.508	0.000
Zipfian	9/1	0.184	0.185	-0.541	0.183	0.546

Figure 6: Speedups of SIEVE and LRU vs Clock.

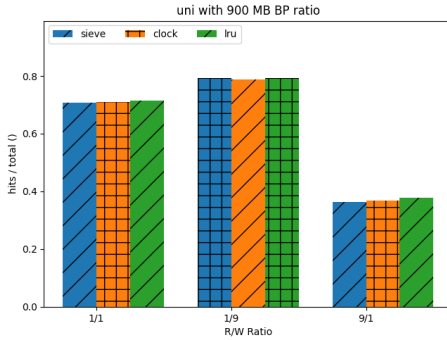


Figure 7: Cache hit ratio of uniform queries with a 900MB BP.

Figures 2, 3, and 6 show these results most clearly. SIEVE outperforms CLOCK in latency across all R/W ratios for the 900 MB bufferpool and uniform query workload, and is within 1 percent performance for all skewed workloads. Despite CLOCK's often higher latency, it maintains a slight edge in cache hit ratios on all buffer sizes with write heavy workloads.

When measuring throughput over different read ratios in Figures 11 and 12, we see little variance in performance between the policies; however, we do see that LRU falls behind the others for uniform queries. This behavior falls in line with Figure 10C of [4] which is the result we were trying to reproduce.

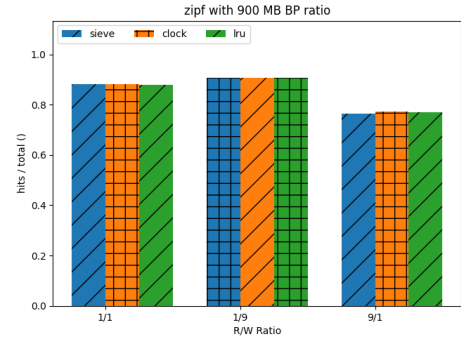


Figure 8: Cache hit ratio of skewed Zipfian queries with a 900MB BP.

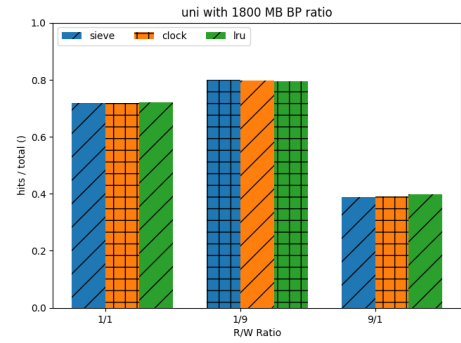


Figure 9: Cache hit ratio of uniform queries with a 1800MB BP.

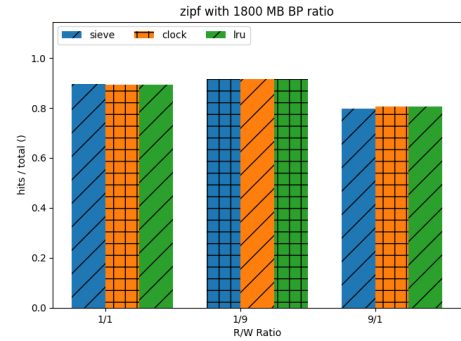


Figure 10: Cache hit ratio of skewed Zipfian queries with a 1800MB BP.

Significantly, the transaction throughput of SIEVE in our experiments exhibited sublinear scaling in-line with CLOCK and LRU (Figure 13), and below the linear scaling observed in Figure 6 of [5]. We have tried to closely replicate the conditions under which the original scaling was seen, but have not seen the same performance characteristics. This could be due to a number of factors, but we believe that it is likely due to the different database environments these tests were run in. The datasets used in [5] are KV-cache traces from real-world systems replayed in a test environment. Postgres

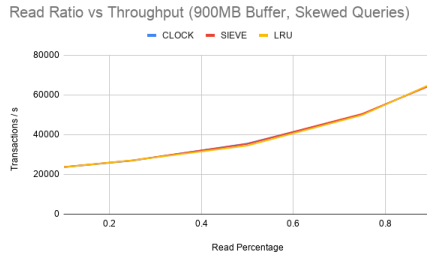


Figure 11: Throughput as R/W ratio increases on skewed queries.

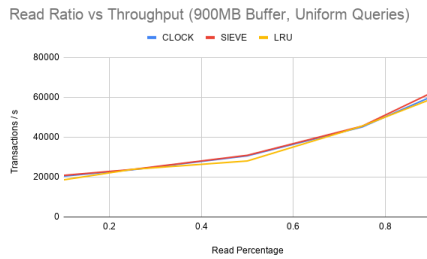


Figure 12: Throughput as R/W ratio increases on uniform queries.

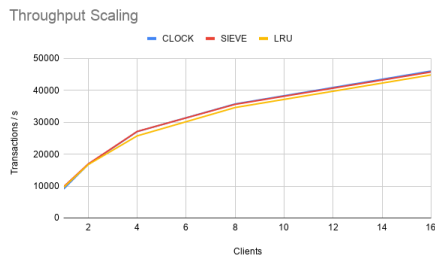


Figure 13: Throughput scaling as clients, threads, and memory increase. We use half as many threads as clients, and scale the bufferpool size proportionally with clients (starting at 900 MB, then 1800 MB, etc.). The workload is skewed, and the workload is mixed 1/1.

is likely to have significantly different indexing structures than a KV-cache. Along with the relational schema, we believe that these differences could hide any performance benefits observed in the context of the original paper. It is also possible that our synthetic benchmarks are not a comparable workload to the traces used in SIEVE. Given more time, we would try and set up this benchmark in a Postgres environment.

Finally, we observed in our testing setup that throughput hit a wall on when the size of the bufferpool hit about %20 of the total data size. We expected to see something more similar to the graph in Figure 10E of [4], in which the workload latency would rapidly decrease as bufferpool size increased.

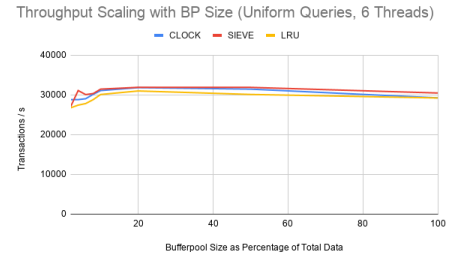


Figure 14: Performance scaling as only buffer size increases for uniform queries on a mixed workload.

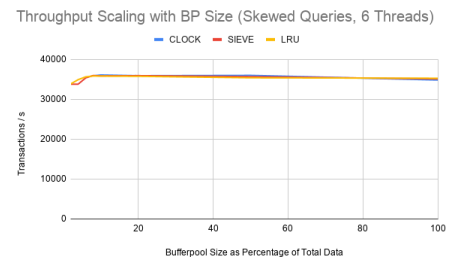


Figure 15: Performance scaling as only buffer size increases for skewed queries on a mixed workload.

To investigate, we verified that cache hit ratio was increasing, and performed the same experiment with a read-heavy workload. Again, we saw the performance gains hit a wall. As we increased the number of threads, the throughput increased, but we did not see a change in the shape of the graph.

Ultimately, we were unable to determine the cause of the bottleneck in this test. As a final effort, we disabled file syncing in postgres to try and remove all disk accesses, but this did not change the shape of our graph.

5 Conclusion

We presented our implementation and findings on SIEVE, a cache eviction primitive within PostgreSQL. We show from our results that SIEVE shows promise on small buffer sizes with better average latency on zipfian and uniform queries, however that advantage does diminish as the buffer size increases. We also note that it is possible that our synthetic benchmarks are not comparable to the traces used in the original paper [5], which could explain why our results showed negligible improvement over CLOCK. The authors of SIEVE explicitly mention that SIEVE would exhibit performance drops on scan workloads. While we acknowledge this gap in our experimentation, future work could explore whether SIEVE's advantages at small buffer sizes generalize to other workloads, and whether a scan-resistant variant of SIEVE could close the performance gap observed in write-heavy and uniform access patterns within PostgreSQL.

References

- [1] Tom Alvaro. 2005. Release 8.1. <https://www.postgresql.org/docs/9.1/release-8-1.html>
- [2] Steven Gong. 2024. Clock Policy. <https://stevengong.co/notes/Clock-Policy>
- [3] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*. USENIX Association, San Francisco, CA. <https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache>
- [4] Tarikul Islam Papon and Manos Athanassoulis. 2023. ACEing the Bufferpool Management Paradigm for Modern Storage Devices. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1326–1339. doi:10.1109/ICDE55515.2023.00106
- [5] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is simpler than LRU: an efficient turn-key eviction algorithm for web caches. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (*NSDI'24*). USENIX Association, USA, Article 68, 18 pages.